

Optimization Techniques

Lab n°4 - Newton's and Quasi-Newton methods

Exercise 1 Write a program that minimizes the function $f(\mathbf{x}) = x_1^2 + 2x_2^2 + \frac{1}{3}x_1^2x_2^4$, using Newton-Raphson's algorithm.

Choose $\mathbf{x}_0 = (0, 1)$ as starting point. Use as stopping criterion $|\nabla f(\mathbf{x}_k)| \leq 10^{-5}$.

Exercise 2 Consider the function

$$f(x_1, x_2) = e^{-x_1^2 - x_2^2} \left(\frac{3}{2}x_2 - 2x_1^2 - x_1^3 + x_1^4 \right).$$

1. Verify (use your program to do that! remind that Matlab and Python compute natively eigenvalues, with the commands `eig(A)` and `numpy.linalg.eig(A)`, respectively) that the point $\mathbf{x}^* = (0.790806, -0.387505)$ satisfies necessary conditions for being a local minimum of the function.

2. Write a program that minimizes the function f , using Newton-Raphson's algorithm.

Choose $\mathbf{x}_0 = (1, 1)$ as starting point. Use as stopping criterion $|\nabla f(\mathbf{x}_k)| \leq 10^{-5}$.

What do you observe?

3. Try to implement a Levenberg-Marquardt modification in order to obtain convergence towards $\mathbf{x}^* = (0.790806, -0.387505)$.

Exercise 3 (If you have time) The aim of this exercise is to compare the performances of BFGS algorithm (pseudocode below) and Newton-Raphson algorithm.

We use as test function the n -dimensional Rosenbrock's function

$$f(\mathbf{x}) = \sum_{i=1}^{n-1} 100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2$$

starting from the initial condition $(1, -1.2, 1, -1.2, \dots)$.

Try with the dimensions $n = 6, 20, 50$.

For **Matlab** users:

```
f=@(x) sum(100 * (x(2:end) - x(1:end-1).^2).^2 + (1 - x(1:end-1)).^2);  
n = choose the dimension;  
x0 = ones(1, n);  
x0(1:2:end) = -1.2;
```

For **python** users, it should (hopefully) be:

```
import numpy as np  
f = lambda x: np.sum(100.0 * (x[1:] - x[:-1]**2)**2 + (1.0 - x[:-1])**2)  
n = choose the dimension;  
x0 = np.ones(n)  
x0[0::2] = -1.2
```

Algorithm 1 BFGS Optimization (Quasi-Newton)

Takes as inputs: an initial point $\mathbf{x}_0$, a function f , an initial inverse Hessian approximation H , tolerance ϵ , max iterations K_{max} , and gradient step h .

```
1:  $dim \leftarrow \text{Length}(\mathbf{x}_0)$ 
2:  $\mathbf{g} \leftarrow \text{grad}(f, \mathbf{x}_0, h)$ 
3:  $I \leftarrow I_{dim}$  ▷ Identity matrix of size  $dim \times dim$ 
4:  $init \leftarrow \|\mathbf{g}\|$ 
5:  $k \leftarrow 0$ 
6: while  $|init| > \epsilon$  and  $k \leq K_{max}$  do
7:    $\mathbf{d} \leftarrow -\mathbf{g}H$  ▷ Compute search direction (row vector)
8:    $\phi(\alpha) \leftarrow f(\mathbf{x}_0 + \alpha\mathbf{d})$ 
9:    $\alpha_{opt} \leftarrow \text{fminsearch}(\phi, 0)$  ▷ Line search for optimal step
10:   $\mathbf{x}_p \leftarrow \mathbf{x}_0 + \alpha_{opt}\mathbf{d}$  ▷ Update point
11:   $\mathbf{g}_p \leftarrow \text{grad}(f, \mathbf{x}_p, h)$  ▷ Compute new gradient
12:   $\Delta\mathbf{g} \leftarrow \mathbf{g}_p - \mathbf{g}$ 
13:   $\Delta\mathbf{x} \leftarrow \mathbf{x}_p - \mathbf{x}_0$ 
14:  if  $\|\Delta\mathbf{g}\| < \epsilon/10000$  then
15:    break ▷ Safety stop if gradient change is negligible
16:  end if
17:   $\rho \leftarrow 1/(\Delta\mathbf{g}\Delta\mathbf{x}^T)$  ▷ Inner product
18:   $H \leftarrow (I - \rho\Delta\mathbf{x}^T\Delta\mathbf{g})H(I - \rho\Delta\mathbf{g}^T\Delta\mathbf{x}) + \rho\Delta\mathbf{x}^T\Delta\mathbf{x}$  ▷ BFGS update matrix
19:   $\mathbf{x}_0 \leftarrow \mathbf{x}_p$ 
20:   $\mathbf{g} \leftarrow \mathbf{g}_p$ 
21:   $init \leftarrow \|\mathbf{g}\|$ 
22:   $k \leftarrow k + 1$ 
23: end while
24: return  $\mathbf{x}_0, H$ 
```
